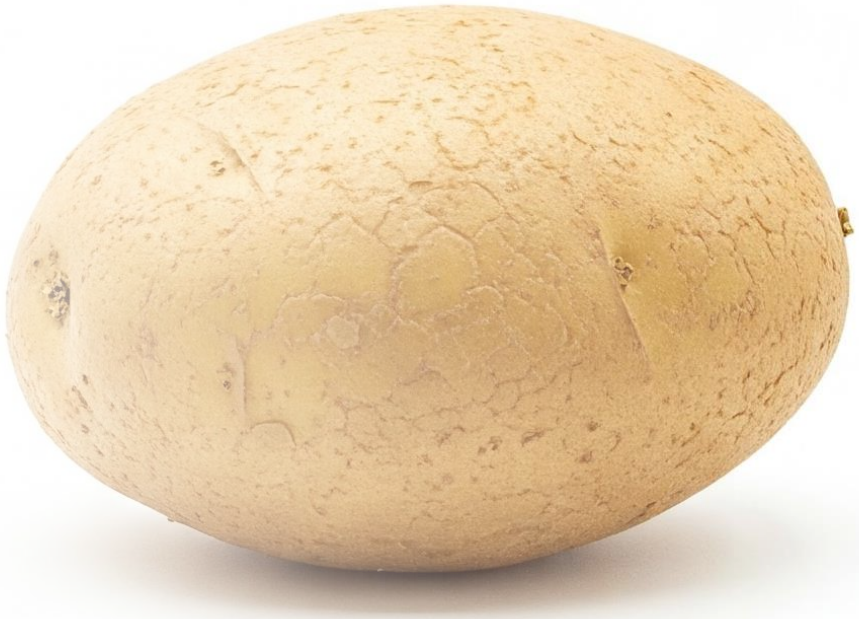
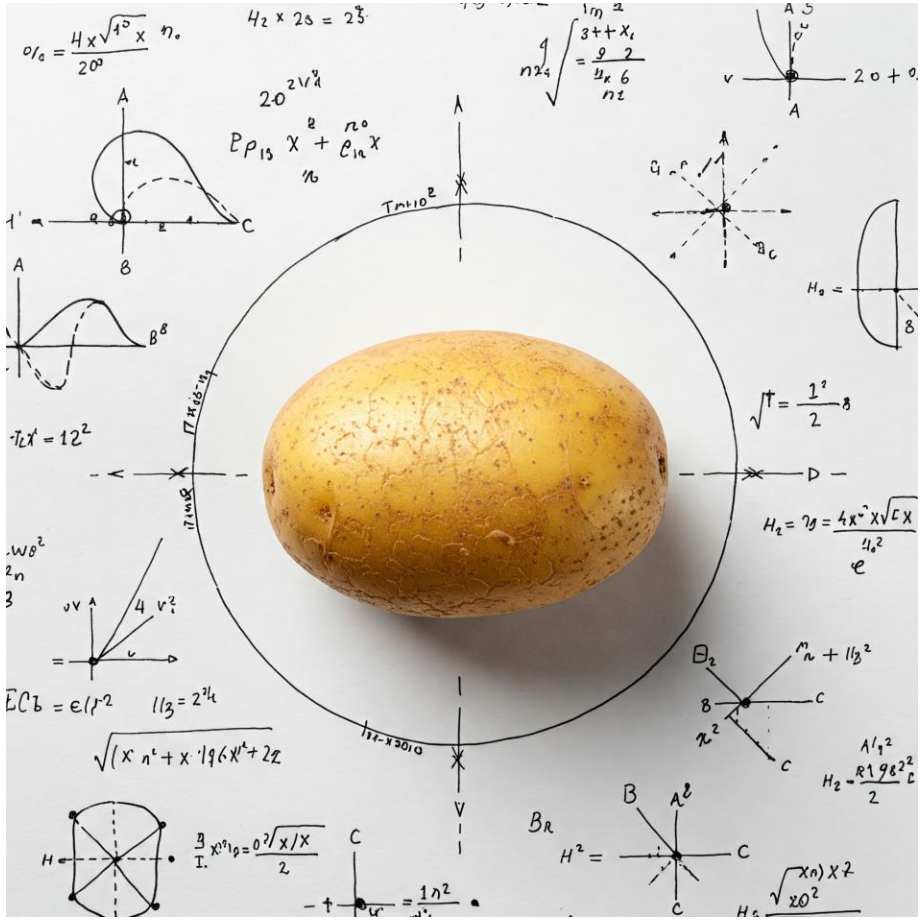
Agenda

- Model i kontekst
- Zależności i asocjacje

Model
i kontekst



Model rzeczywistości



```
public class Potato {
  private String species;
  private String variety;
  private double mass;
  private double size;
  private double volume;
  private double density;
  private double waterContent;
  private double starchContent;
  private double proteinContent;
  private double fatContent;
  private boolean isSprouting;
  /* ... */
}
```

Kontekst modelu

```
public class Laboratory {
  Potato potatoes[];
  public Laboratory(int size) {
    potatoes = new Potato[size];
  }
  public void plantPotato(int index) {
    potatoes[index] = new Potato(/* ... */);
  }
  public void addNutritients(Potato p, Nutritients[] n)
  {
    /* ... */
  }
  public GrowthParameters getGrowthParameters(Potato p)
  {
    /* ... */
  }
}
```

Dopasowanie modelu do kontekstu

```
public class Potato {
  private boolean rotten;
  public boolean isRotten() {
    return rotten;
  }
  /* ... */
}
public class Kitchen {
  public static void useForDish(Potato potato) {
    if (potato.isRotten())
      return;
    else { /* ... */ }
  }
}
```

Dopasowanie modelu do kontekstu

```
public class Potato {
  private static int price;
  private final float weight;
  public static int getPrice() {
    return price;
  }
  public static void setPrice(int price) {
    Potato.price = price;
  }
  public float getWeight() { return weight; }
  public Potato(float weight) {
    this.weight = weight;
  }
}
```

złe

Dopasowanie modelu do kontekstu

```
public class Stall {
  private int money = 200_00;
  private Potato[] potatoes;
  /* ... */
  public void sell(Potato[] soldPotatoes) {
    float totalWeight = 0;
    for (int i = 0; i < potatoes.length; i++)
      for (Potato soldPotato : soldPotatoes)
        if (potatoes[i] != null && potatoes[i] == soldPotato) {
          potatoes[i] = null;
          totalWeight += soldPotato.getWeight();
        }
        money += Math.round(Potato.getPrice() * totalWeight);
    }
}
```

Reprezentacja zbiorcza

```
public class Potatoes {
  private final int price;
  private float weight;
  public Potatoes(int price, float weight) {
    this.price = price;
    this.weight = weight;
  }
  public int getPrice() { return price; }
  public float takePotatoes(float weightToTake) {
    float takenWeight = Math.min(weight, weightToTake);
    weight = weight - takenWeight;
    return takenWeight;
  }
}
```

Reprezentacja zbiorcza

```
public class Stall {
  int money = 200_00;
  Potatoes potatoes;
  /* ... */

  public void sell(float weight) {
    float takenWeight = potatoes.takePotatoes(weight);
    money += Math.round(Potato.getPrice() * takenWeight);
  }
}
```

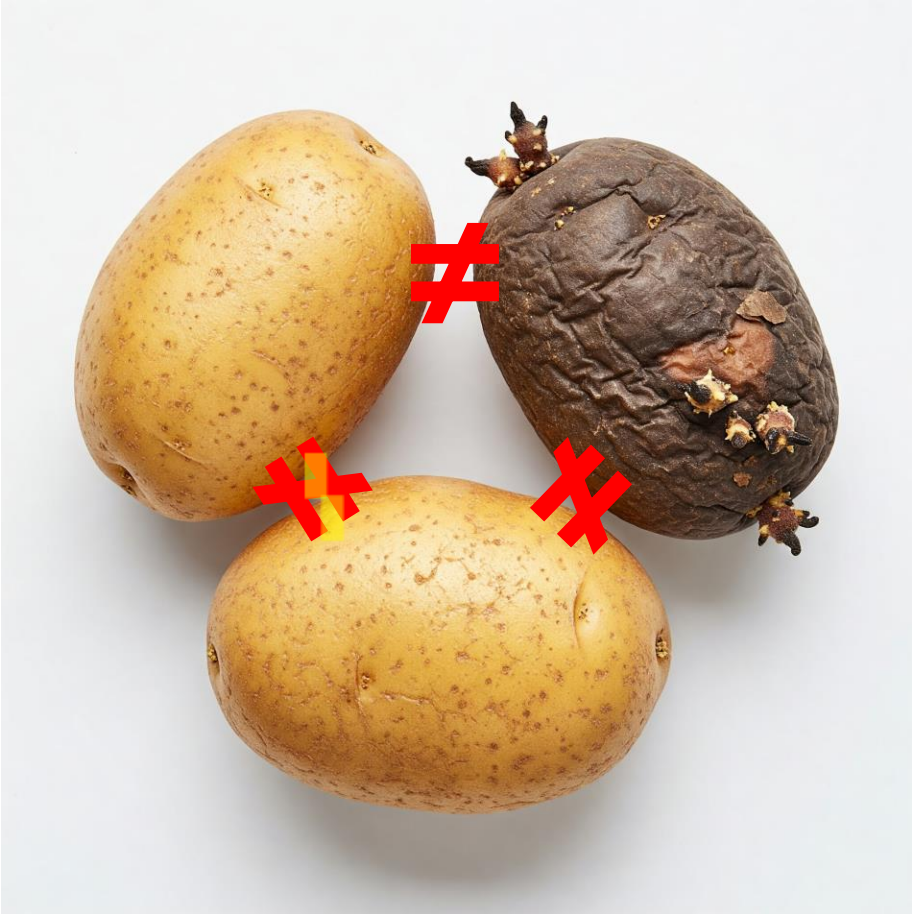
Brak reprezentacji obiektowej

```
int potatosWeight = 2000;
```

Encja

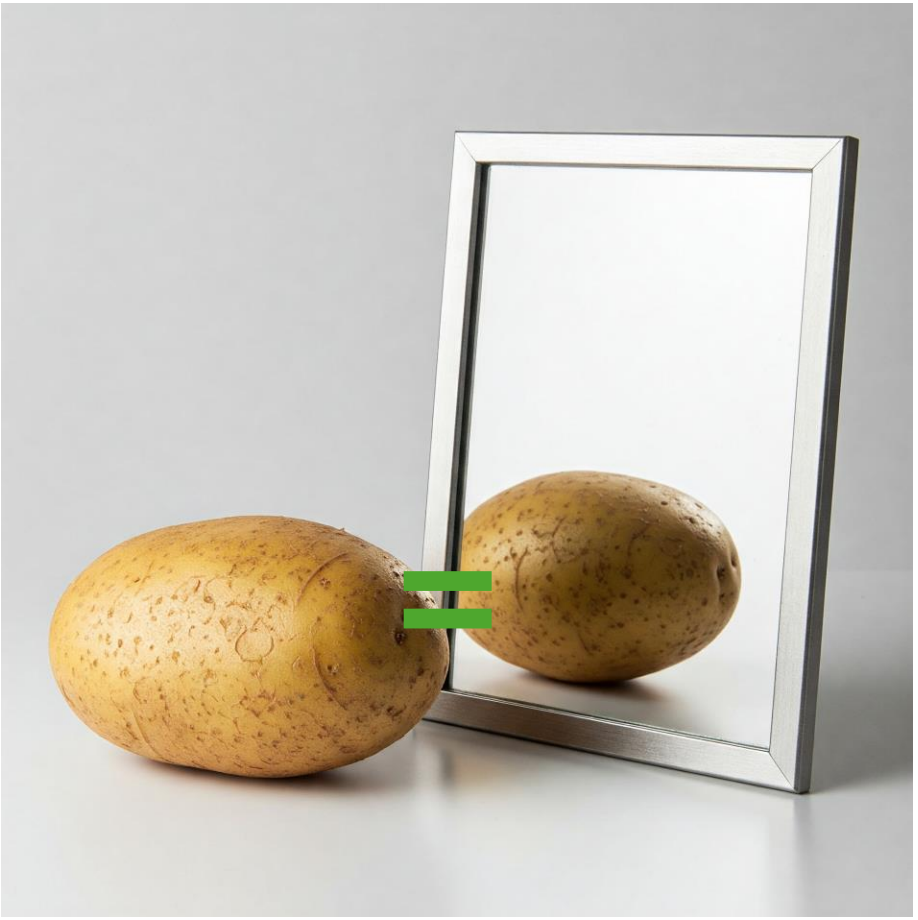
```
public class Potato {
  private boolean rotten = false;
  public boolean isRotten() { return rotten; }
  public void rot() { rotten = true; }
}

Potato goodPotato = new Potato();
Potato nicePotato = new Potato();
Potato badPotato = new Potato();
badPotato.rot();
```

Encja



```java
public class Potato {
  private boolean rotten = false;
  public boolean isRotten() { return rotten; }
  public void rot() { rotten = true; }
}

Potato goodPotato = new Potato();
Potato nicePotato = new Potato();
Potato badPotato = new Potato();
badPotato.rot();

System.out.println(goodPotato == badPotato);  // false
System.out.println(nicePotato == badPotato);  // false
System.out.println(goodPotato == nicePotato); // false
```

Encja



```
public class Potato {
  private boolean rotten = false;
  public boolean isRotten() { return rotten; }
  public void rot() { rotten = true; }
}

Potato goodPotato = new Potato();
Potato sameGoodPotato = goodPotato;



System.out.println(goodPotato == sameGoodPotato);
  // true
```

Typ wyliczeniowy

```
public class Potato {
  public enum Variety { WAXY, MEDIUM, STARCHY }

  private Variety variety;
  private boolean rotten = false;
  public boolean isRotten() { return rotten; }
  public void rot() { rotten = true; }

  public Potato(Variety variety) {
    this.variety = variety;
  }
}

Potato starchyPotato = new
  Potato(Potato.Variety.STARCHY);
```

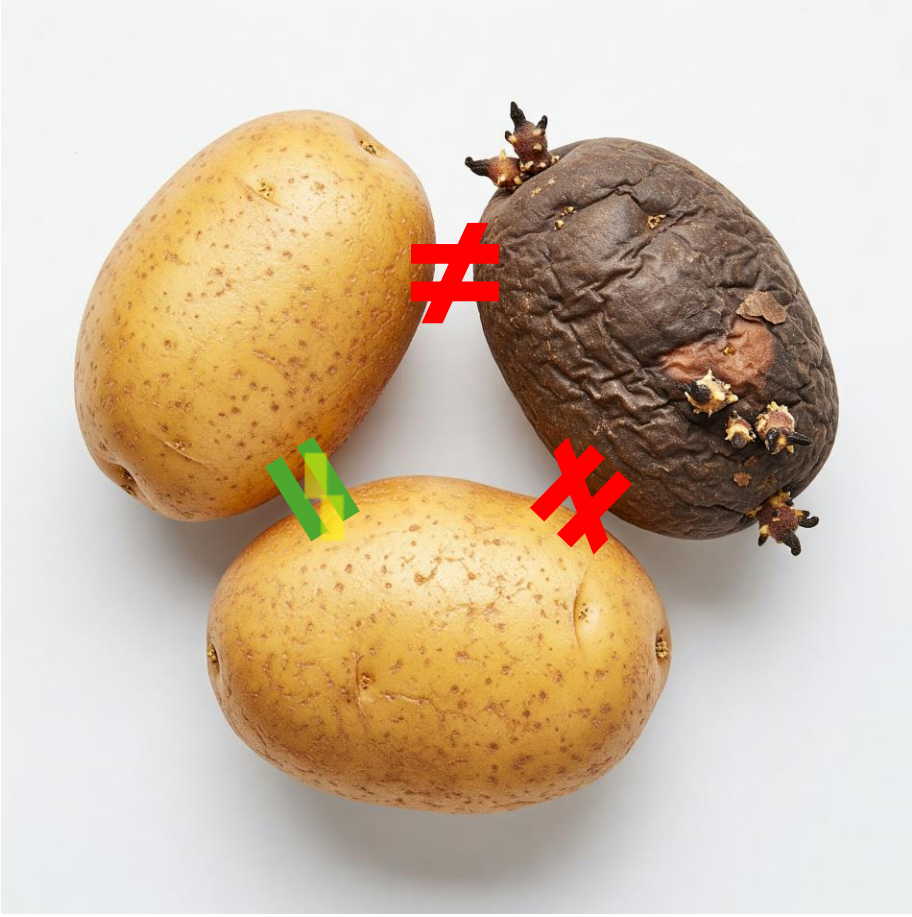
Obiekt jako zestaw cech

```
public class Potato {
  public enum Variety { WAXY, MEDIUM, STARCHY }

  public final Variety variety;
  public final boolean rotten;
  public Potato(Variety variety, boolean rotten) {
    this.variety = variety;
    this.rotten = rotten;
  }
}
```

Porównanie cech obiektów

```
public class Potato {
  public enum Variety { WAXY, MEDIUM, STARCHY }

  public final Variety variety;
  public final boolean rotten;
  public Potato(Variety variety, boolean rotten) {
    this.variety = variety;
    this.rotten = rotten;
  }
  public boolean equals(Object o) {
  if (!(o instanceof Potato potato)) return false;
    return rotten == potato.rotten
        && variety == potato.variety;
  }
}
```

Obiekt wartościowy



```
Potato goodPotato = new Potato(
                        Potato.Variety.STARCHY, false);

Potato nicePotato = new Potato(
                        Potato.Variety.STARCHY, false);

Potato badPotato = new Potato(
                        Potato.Variety.STARCHY, true);


System.out.println(goodPotato == badPotato); // false

System.out.println(nicePotato == badPotato); // false

System.out.println(goodPotato == nicePotato); // false


System.out.println(goodPotato.equals(badPotato));
  // false

System.out.println(nicePotato.equals(badPotato));
  // false

System.out.println(goodPotato.equals(nicePotato));
  // true
```

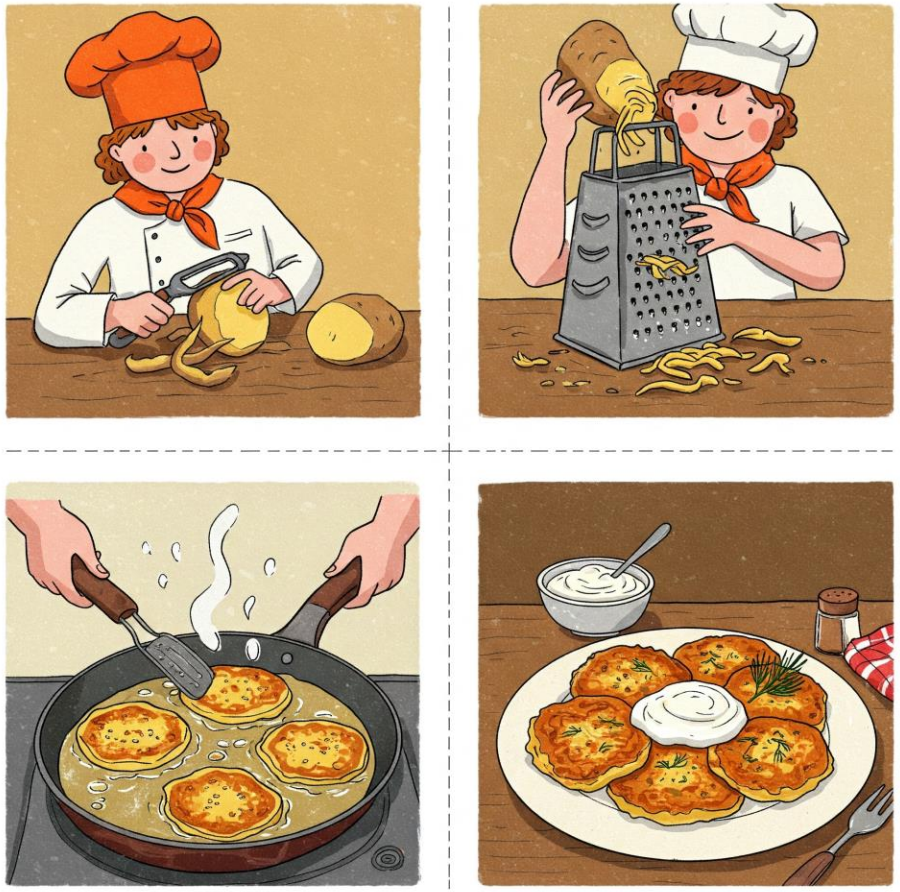
Rekord

```java
public record Potato(
    Variety variety,
    boolean rotten
  ) {
  public enum Variety {
    WAXY, MEDIUM, STARCHY
  }
}
```

=

```java
public class Potato {
  public enum Variety { WAXY, MEDIUM, STARCHY }

  public final Variety variety;
  public final boolean rotten;
  public Potato(Variety variety, boolean rotten) {
    this.variety = variety;
    this.rotten = rotten;
  }
  public boolean equals(Object o) {
  if (!(o instanceof Potato potato)) return false;
    return rotten == potato.rotten
        && variety == potato.variety;
  }
}
```

Obiekt w łańcuchu przekształceń



```
class PeeledPotato {
  public PeeledPotato(Potato potato) { /* ... */ }
}
class GratedPotato {
  public GratedPotato(PeeledPotato potato) { /* ... */ }
}
class PotatoPancake {
  public PotatoPancake(GratedPotato potato) { /* ... */ }
}
public PeeledPotato peel(Potato potato) {
  return new PeeledPotato(potato);
}
public GratedPotato grate(PeeledPotato peeledPotato) {
 return new GratedPotato(peeledPotato);
}
public PotatoPancake fry(GratedPotato gratedPotato) {
  return new PotatoPancake(gratedPotato);
}
```

String jako obiekt wartościowy

```cpp
std::string s = "Ala ma kota";
s.replace(0, 1, "O");

std::cout << s << std::endl; // Ola ma kota
```

```java
String s = "Ala ma kota";
String m = s.replace("A", "O");

System.out.println(s); // Ala ma kota
System.out.println(m); // Ola ma kota
```

Porównywanie napisów

```cpp
std::string a = "Ala ma kota";
std::string b = "Ala ma kota";
std::cout << (a == b) << std::endl;
// 1

std::string &refA = a;
std::string &refB = b;
std::cout << (refA == refB) << std::endl;
// 1

std::string *ptrA = &a;
std::string *ptrB = &b;
std::cout << (ptrA == ptrB) << std::endl;
// 0
```

```java
String a = "Ala ma kota";
String b = "Ala ma kota";
System.out.println(a == b);
// true
```

Cache'owanie napisów

```cpp
std::string a = "Ala ma kota";
std::string b = "Ala ma kota";
std::cout << (a == b) << std::endl;
// 1

std::string &refA = a;
std::string &refB = b;
std::cout << (refA == refB) << std::endl;
// 1

std::string *ptrA = &a;
std::string *ptrB = &b;
std:cout << (ptrA == ptrB) << std::endl;
// 0
```

```java
String a = "Ala ma kota";
String b = "Ala ma kota";
System.out.println(a == b);
// true

String prefix = "Ala ma ";
String c = prefix + "kota";
String d = prefix + "kota";
System.out.println(c == d);
// false
```

Porównywanie wartości napisów

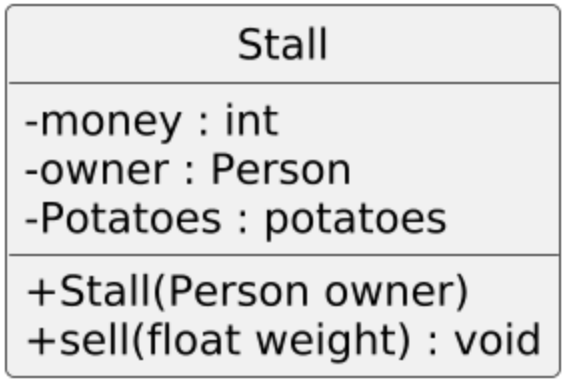
```cpp
std::string a = "Ala ma kota";
std::string b = "Ala ma kota";
std::cout << (a == b) << std::endl;
// 1

std::string &refA = a;
std::string &refB = b;
std::cout << (refA == refB) << std::endl;
// 1

std::string *ptrA = &a;
std::string *ptrB = &b;
std::cout << (ptrA == ptrB) << std::endl;
// 0
```

```java
String a = "Ala ma kota";
String b = "Ala ma kota";
System.out.println(a.equals(b));
// true

String prefix = "Ala ma";
String c = prefix + "kota";
String d = prefix + "kota";
System.out.println(c.equals(d));
// true
```

Klasa z polami i metodami

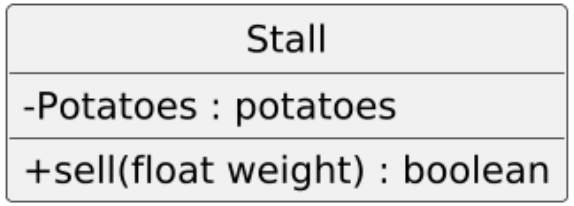
```
public class Stall {
  private int money = 200_00;
  private Person owner;
  private Potatoes potatoes;

  public Stall(Person owner) {
    this.owner = owner;
    this.potatoes = new Potatoes(1_99, 500);
  }
  public void sell(float weight) {
    float takenWeight = potatoes.takePotatoes(weight);
    money = Math.round(potatoes.getPrice() *
  takenWeight);
  }
}
```

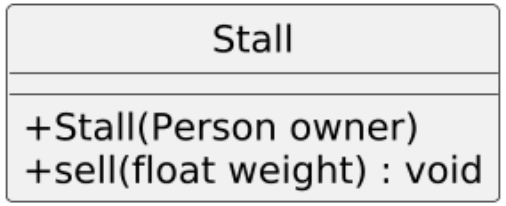
Schemat klasy w UML




```java
public class Stall {
  private int money = 200_00;
  private Person owner;
  private Potatoes potatoes;

  public Stall(Person owner) {
    this.owner = owner;
    this.potatoes = new Potatoes(1_99, 500);
  }
  public void sell(float weight) {
    float takenWeight = potatoes.takePotatoes(weight);
    money = Math.round(potatoes.getPrice() *
  takenWeight);
  }
}
```

Uproszczenia schematu



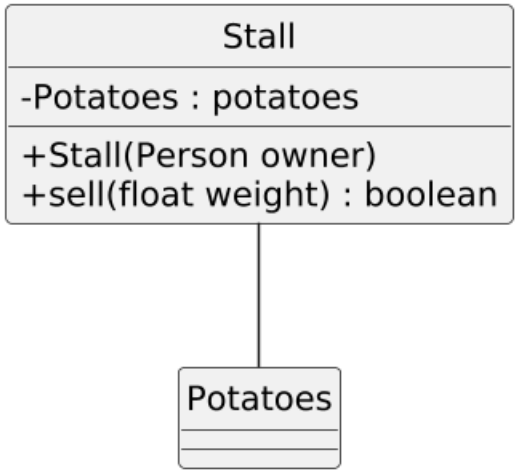


```java
public class Stall {
  private int money = 200_00;
  private Person owner;
  private Potatoes potatoes;

  public Stall(Person owner) {
    this.owner = owner;
    this.potatoes = new Potatoes(1_99, 500);
  }
  public void sell(float weight) {
    float takenWeight = potatoes.takePotatoes(weight);
    money = Math.round(potatoes.getPrice() *
  takenWeight);
  }
}
```

Zależności i asocjacje

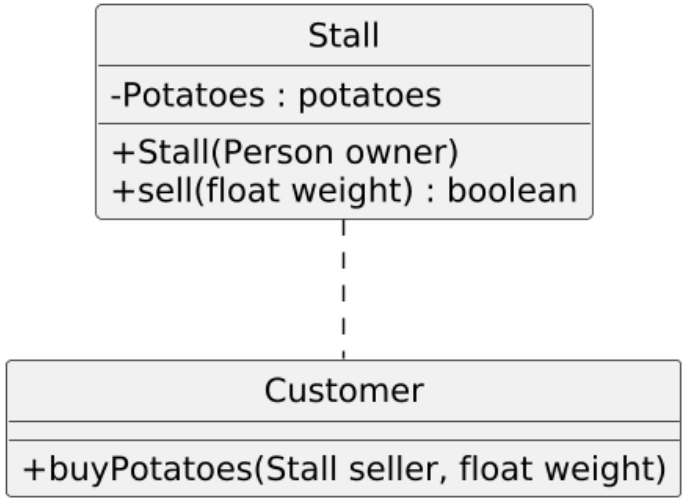
Asocjacja

```
public class Stall {
  /* ... */
  Potatoes potatoes;
}
```



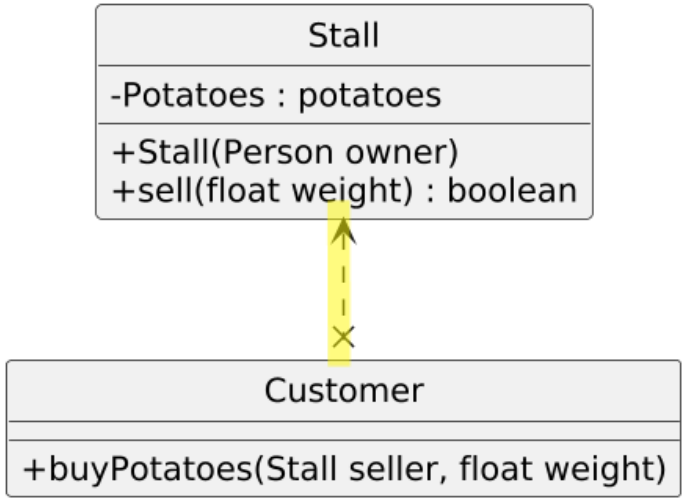
Zależność

```
public class Customer {
  /* ... */
  public void buyPotatos(Stall seller, float weight) {
    seller.sell(weight);
  }
}
```



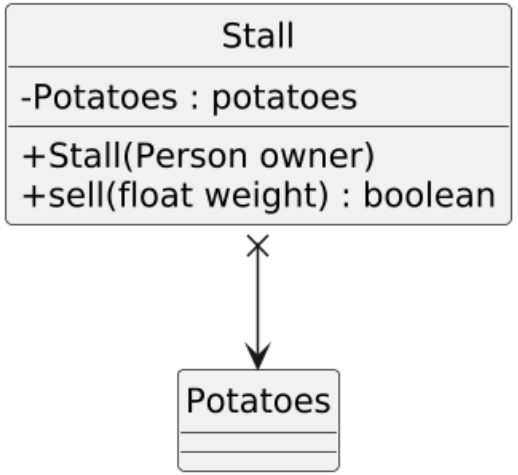
Nawigowalność

```
public class Customer {
  /* ... */
  public void buyPotatos(Stall seller, float weight) {
    seller.sell(weight);
  }
}
```

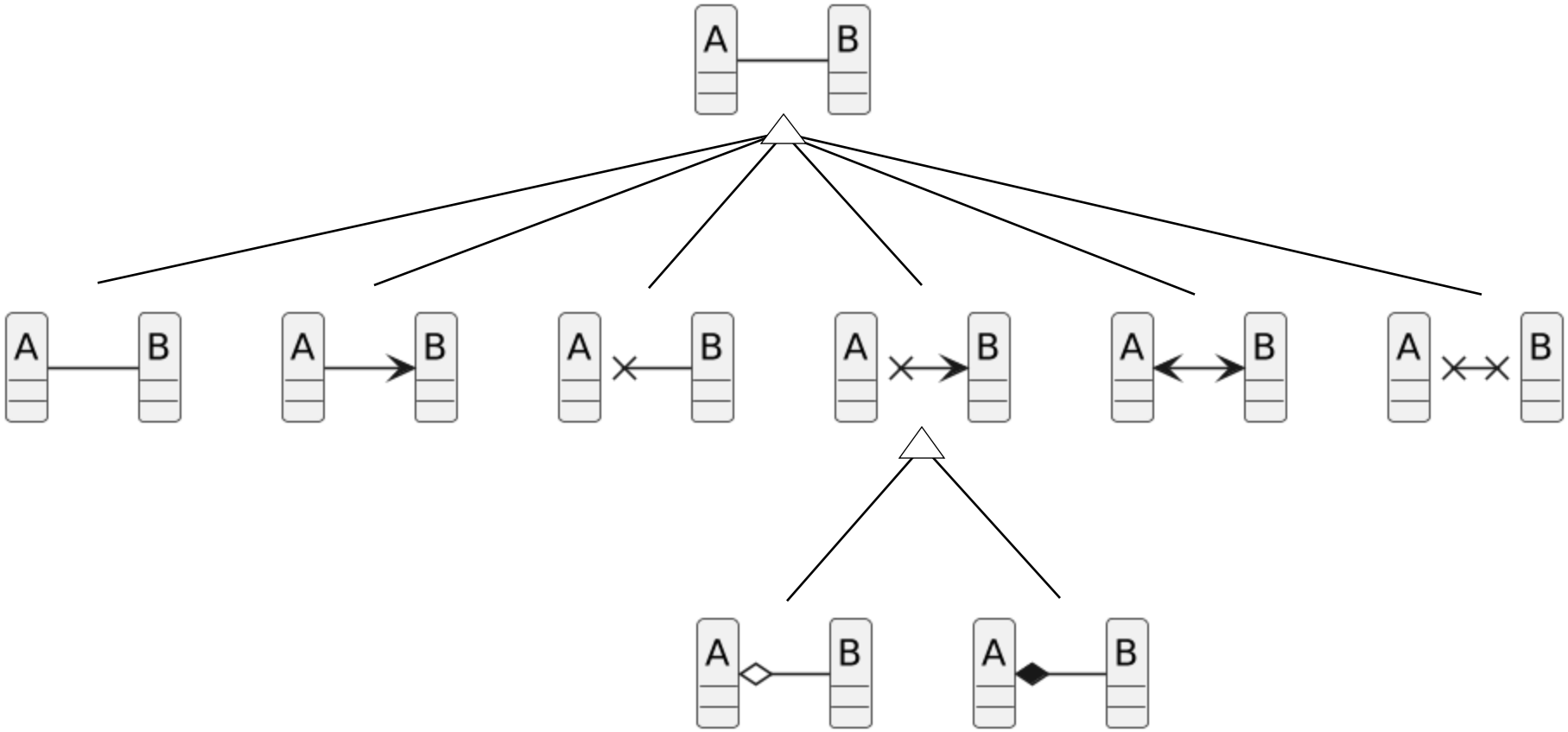


Nawigowalność

```
public class Stall {
  /* ... */
  Potatoes potatoes;
}
```



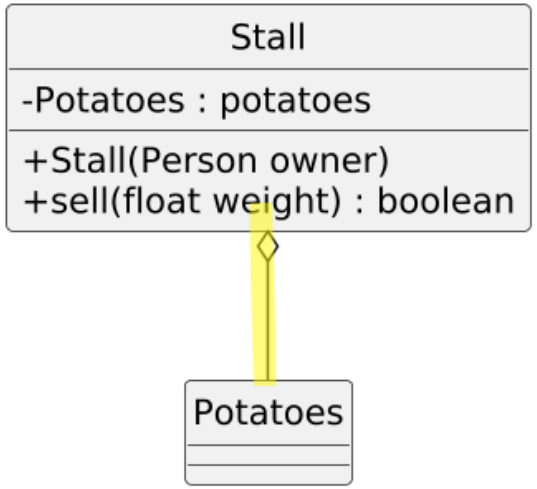
Rodzaje asocjacji



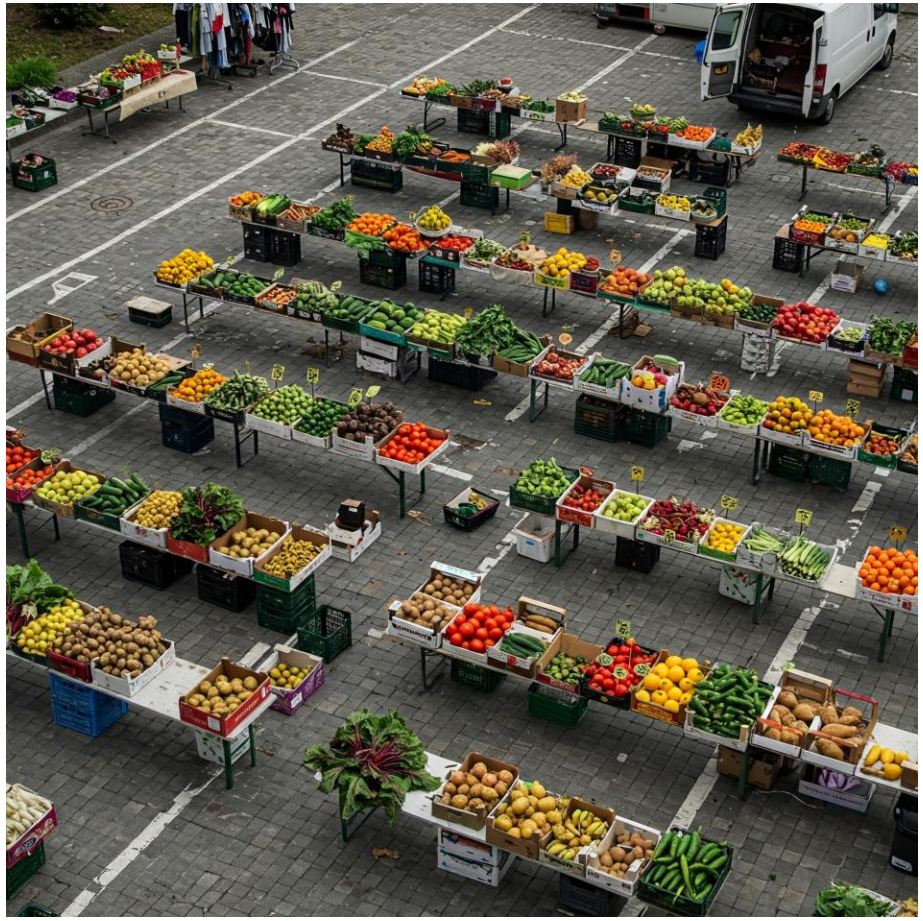
Agregacja

- Binarna.
- Asymetryczna.
- Przechodnia.
- Współdzielona.

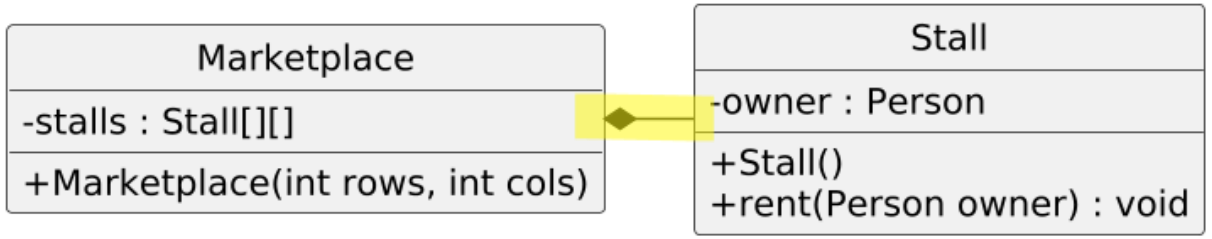
```
public class Stall {
  /* ... */
  Potatoes potatoes;
}
```

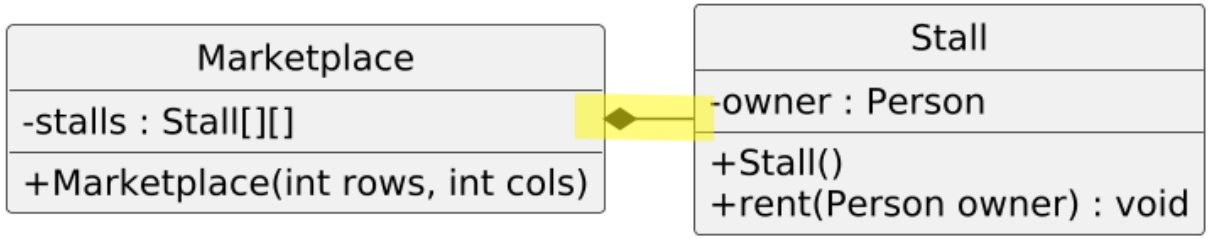


Kompozycja



```java
public class Marketplace {
  private Stall[][] stalls;
  public Marketplace(int rows, int columns) {
  stalls = new Stall[rows][columns];
  for (int i = 0; i < rows; i++) {
    for (int j = 0; j < columns; j++) {
      stalls[i][j] = new Stall();
  }
```



Kompozycja

- Binarna.
- Integralna.
- Uzależniona od kompozytu.